Blender 白模预演插件：剧本解构粒度设计研究报告

> 面向”用 LLM 解构剧本 → 驱动 Blender 白模预演”的插件开发者。核心议题：是否应该规划剧本到实现的步骤、每个分镜应拆解到什么粒度、以及如何设计 LLM 与插件之间的数据契约。

TL;DR（直接回答）

应该规划实现步骤——剧本是”非空间化”的文本，Blender 是”纯数值化”的场景，两者之间存在巨大的语义鸿沟，不设计中间层而指望 LLM 一步到位输出可用的场景数据或 bpy 代码，是目前学术界（SceneCraft、3D-GPT、MovieAgent 等）都已验证过的失败路径。但**你列举的拆解维度（角色行动关键帧、旋转、道具关键帧、前景/中景/后景、灯光、运镜关键帧）混合了”对象”与”通道”两个正交维度，并且把”关键帧”这个实现细节当成了 LLM 的解构单元，这是需要修正的核心问题**。推荐的粒度甜区是：LLM 输出**语义级、可枚举的决策**（谁、在哪、做什么动作、相机用什么运镜原语、景别、时机与情绪），帧号、坐标、旋转角度等连续数值全部由插件侧的确定性求解器生成。一句话：**LLM 做”分镜师”，求解器做”执行”，Blender API 做”落地”**——**关键帧是求解器的输出，不是 LLM 的输出**。

---

1. 为什么必须规划实现步骤：剧本到场景的语义鸿沟

剧本是人类为”有具身经验的读者”写的压缩格式。“甲走进房间，在桌边坐下”这句话，人类分镜师能瞬间补全：门在哪、桌子多高、走几步、用几秒、镜头跟不跟。但对生成系统而言，这句话里**没有任何空间坐标、没有任何时间度量、没有任何相机信息**——它需要被解压缩成一个包含数百个数值参数的场景状态。这个解压过程如果交给 LLM 一步完成，等于要求它同时做编剧、分镜师、layout 艺术家、动画师和绑定师，而每一步的错误都会向下游放大。

学术界的证据相当一致。SceneCraft（ICML 2024）让 LLM 直接为 Blender 写 Python 脚本生成场景，其关键发现是：必须先在中间层建立一个**关系场景图（scene graph）**作为蓝图，再由代码把空间关系翻译成数值约束交给求解器求值，直接让 LLM 预测资产的位置、朝向和尺寸会因”组合空间爆炸”而失败；其对照基线 BlenderGPT 因为缺少这个中间抽象层，只能完成”移动资产、换材质”这类简单指令（SceneCraft, ICML 2024）。在视频生成一侧，LAMP 用 3D 包围盒轨迹作为抽象层，并明确指出”这种抽象镜像了电影工业中标准的预演与走位（previsualization and blocking）实践——用简化几何代理来推演场面调度、构图和相机运动，然后再渲染”（LAMP）。MovieAgent 则把整个剧本到成片的过程拆成导演、编剧、分镜、选景四个 LLM 角色的**层级化 CoT 推理**，证明分层规划在剧本忠实度、角色一致性和叙事连贯性上优于单步生成（MovieAgent）。

工业侧的逻辑相同。传统剧本拆解（script breakdown）本身就是一套成熟的”中间表示”：AD 团队逐场标记角色、道具、车辆、特技、特效等元素，这套 Color-Coded Element System 是整个排期与预算系统的输入，“拆解的两层结构中，标记说明场景需要什么，意图说明它为什么重要”（Storyflow）。而预演阶段的行业共识同样是”故事优先于精度”——粗糙但清晰的画面永远胜过拖慢迭代的精致画面，且必须使用真实世界尺度以保证镜头行为可复现（DrawStory）。**规划实现步骤不是可选项，而是把”一次性不可控生成”改造成”分层可控生成”的必要手段**：每一层都可以独立校验、独立重生成、独立被人接管，这是白模预演工具与”AI 玩具”的分水岭。

2. 解构到什么粒度：三条判定原则

粒度问题本质上是**“LLM 与确定性代码之间的职责边界画在哪”**的问题。综合学术方案、Blender 数据模型与结构化输出工程实践，可以提炼出三条可操作的判定原则。

2.1 原则一：LLM 只输出”有限词表能表达的决策”

检验某个字段该不该交给 LLM 的最简标准是：**它的取值空间能否被枚举成一张词表**。景别（ECU/CU/MCU/MS/MLS/LS/ELS）可以枚举，运镜类型（pan/tilt/dolly/truck/pedestal/arc/orbit/handheld/static）可以枚举，动作动词（walk_to/sit/stand/pick_up/look_at/exit_frame）可以枚举，情绪标签、昼夜、前中后景层次也都可以枚举——这些交给 LLM，且用 JSON Schema 的 `enum` 约束死。反过来，“第 36 帧时角色 X 坐标为 (3.2, -1.5, 0)、

绕 Z 轴旋转 1.57 弧度”这种连续数值**不能枚举**，LLM 生成它们既没有可靠依据也无法被语义校验，纯粹是幻觉温床。LAMP 的 DSL 正是这样设计的：相机运动被编码为”运动原语 + 键值修饰符”（如平移控制 lat/vert/depth、旋转控制 yaw/pitch/roll、时序修饰 speed_fast/ease_in/jitter_low），符号程序再被**确定性地映射**为 3D 轨迹 (LAMP)。

这条原则在你最初的问题清单里对应一个关键修正：**“角色行动关键帧”“运镜关键帧”不应该是 LLM 输出的东西**。LLM 应该输出”甲在 beat 2 从门口走到桌边（walk_to，目标 = 桌边标记点，节奏 = 快）“和”dolly_in，慢速，主体 = 甲，景别从 MS 收到 CU”，然后由你的求解器根据场景几何（门口坐标、桌边坐标、角色步速假设）算出具体的帧号与关键帧数值。帧号、坐标、弧度是求解器的**输出**，不是 LLM 的输出——这是整个设计里最重要的一条边界。

2.2 原则二：粒度单位必须”可独立校验、可独立重生成”

结构化输出的工程实践表明，schema 越复杂、嵌套越深、单次产出越长，出错率越高；社区实测即使开启 strict 模式，复杂 schema 下模型仍会偶发越界（幻觉出不存在的枚举值、把父级属性写成子级属性），典型可靠性在”90 次里对 89 次”的量级 (OpenAI 社区实测)。工程界的对策共识是：schema 保持聚焦与最小化、优先用枚举代替自由字符串、嵌套深度控制在 5–10 层以内、并且把 schema 当 API 契约一样做版本管理 (Towards Data Science, TrueFoundry)。

映射到你的插件：**解构的原子单位应该是”镜”（shot），而不是”场”（scene），更不是”全片”**。一个镜头一份 JSON 契约，意味着单个镜头生成失败只需要重试这一个镜头，用户改了一个镜头的走位只需要重跑这一个镜头的求解器，校验器也只需要检查这一份小而扁平的结构。这与真实分镜师的工作单元一致——AniMaster 基于 Bordwell 的电影形式体系，把”镜头”定义为视频空间的基本单元，并把视觉叙事的关键变量组织为横跨四个子系统的 9 个参数：摄影（景别、角度、运动）、场面调度（构图、灯光、色调）、剪辑（转场、时长）、声音 (AniMaster)。AI 分镜工具的真实用户行为也印证了”逐镜头迭代”的合理性：导演们的实际工作流是”生成初版 → 逐帧反应式修改（这个角色应该在右边、光应该更硬）→ 1–3 轮收敛”，而不是一次生成整片 (StoryBirdie)。

2.3 原则三：白模预演的本质是 blocking，不是动画

很多人做这类插件时会不自觉地滑向”让 LLM 生成完整动画”，但预演（previs）的产品定位决定了你只需要做到**动画流程中的 blocking 那一层**。3D 动画的标准流程是 Planning（故事板/动态故事板）→ Blocking（大动作与节奏，stepped keys）→ Refining（次级运动、曲线打磨）(Blender For Dummies, 4th ed.)，而 blocking 的方法论非常明确：先放**接触键**（contact，脚踩地、手抓物的锚点帧），再放**极限姿势**（extremes，每个叙事拍点最有信息量的姿势），最后只在观众需要澄清方向时才补过渡姿势；整个 blocking 阶段刻意保持 stepped 插值，就是为了排除软件自动插值的干扰、快速验证节奏 (Cursa 动画教程)。动作戏预演的实践甚至更进一步：直接用方块等代理体先做完整段落的运动和惯性传递，“因为这些镜头需要大量迭代甚至推翻重来，速度和回放流畅性是第一位的” (Kiel Figgins)。

这条原则给你的粒度设计画了一个”下限天花板”：**你的解构只需要覆盖到 blocking 所需的信息量**——每个角色在每个镜头内的 2–5 个关键拍点（beat）、每个拍点的位姿语义与时机、相机在镜头内的起止构图与运动方式。不需要表情、不需要手指、不需要次级运动、不需要曲线手柄。这不仅降低了 LLM 的负担，也符合预演的用途：动态故事板的核心是”每块板按真实时长停留、节奏可检验”，运动只是可选项 (Storyflow Animatic Guide)。

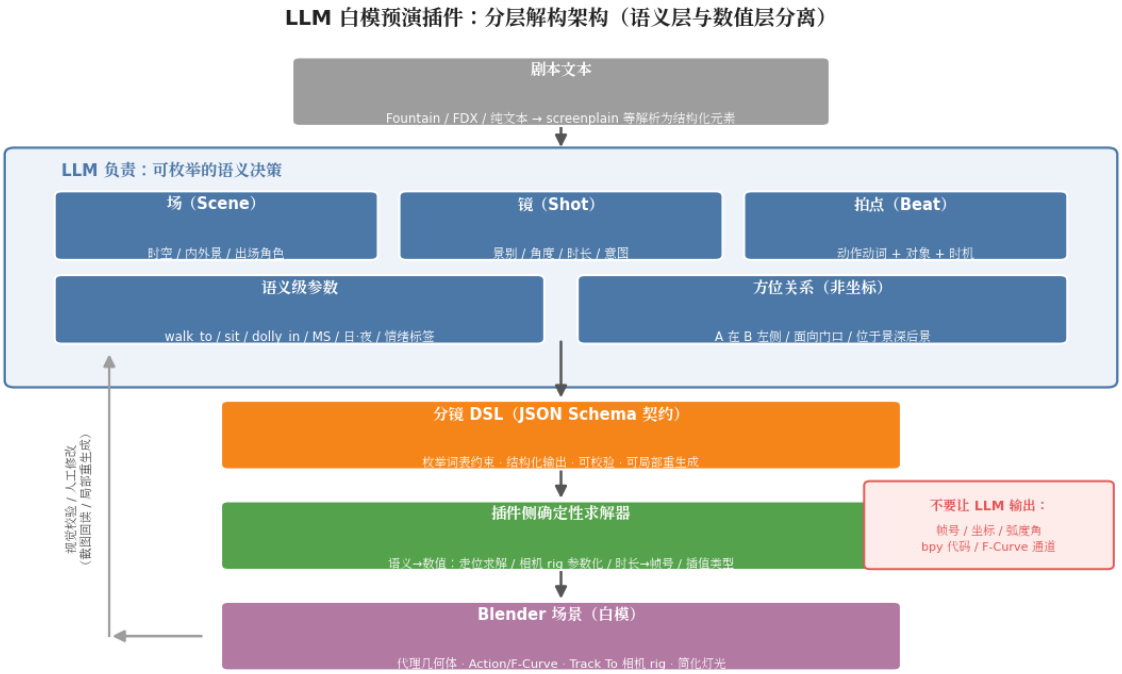


Figure 1: LLM 白模预演插件的分层解构架构

3. 对你提出的拆解维度的评估与修正

你提出的清单是：角色行动关键帧、角色行动旋转、道具关键帧、前景、中景、后景、灯光、运镜关键帧。这个清单的方向感是对的（覆盖了演员、道具、空间层次、灯光、相机五类要素），但存在三个结构性问题。

你的维度	问题诊断	修正方案
角色行动关键帧	“关键帧”是实现层概念，LLM 不该产帧号；且”行动”应拆为”拍点 + 动作动词 + 目标”	改为 **角色拍点表**：每拍点 = 动作动词（枚举）+ 目标锚点 + 相对时机 + 节奏修饰
角色行动旋转	旋转不是独立维度，它是”朝向关系”的数值化结果；LLM 应输出”面向谁/背对哪”	并入拍点的 **朝向字段**：face_to（目标）/ facing（screen_left/right/camera/away）
道具关键帧	道具只在”被交互”时才需要动；独立列维度会诱导给静态道具也打关键帧	改为 **道具交互事件**：attach/detach/位移，挂接到角色拍点的时间线上
前景 / 中景 / 后景	这是布景的**纵深分层**，属于静态 staging，方向正确，但需要锚定到”相机视角”而非世界坐标	保留，作为 **staging 轨道**的 depth_layer 字段（fg/mid/bg），配合与相机的相对关系
灯光	白模阶段通常**不需要精细灯光**；列成一级维度会诱导 LLM 输出灯具参数	降级为**极简语义字段**：time_of_day（日/夜/黄昏）+ mood（明快/阴郁）+ 可选主光方向，由预设模板落地

Table 1 – continued

你的维度	问题诊断	修正方案
运镜关键帧	同”关键帧”问题；运镜应表达为”原语 + 修饰符”	改为 **相机运动原语**：dolly_in/pan/arc/orbit…+ speed/subject/framing_from→to

表中反复出现的模式是：**你的清单把”谁在动”（对象轴）和”动的什么通道”（属性轴）混在了一维**。专业工具的数据模型从来都是两条正交的轴：纵向是对象（角色/道具/相机/灯），横向是通道（位移/旋转/属性），而 LLM 层的解构只应该触及”对象 + 语义动作”，通道的展开交给求解器。动画行业里 blocking 阶段的拍点描述（如”帧 01 注意到物体 / 帧 08 决策预备 / 帧 14 手接触物体 / 帧 26 拿走”）恰好就是这个粒度的天然样板 (Cursa 动画教程)。

另一个需要警惕的是”为了完备而完备”。AI 分镜工具的实践者明确警告过度拆解的副作用：“过度绘制静态场景反而会束缚动画师的解读空间——只板出关键情绪拍点即可” (StoryBirdie)。对你的插件而言同理：**解构粒度每细一档，LLM 出错面、token 成本、用户审核成本都同步上涨，而预演的收益并不上涨**。镜头数量大、单镜信息少，才是可规模化的组合。



Figure 2: 解构粒度光谱：过粗、甜区与过细

4. 推荐的分层解构模型

综合以上证据，推荐一个四级层次 + 五条轨道的解构模型。层次回答”剧本如何被切分”，轨道回答”每个镜头内部分几个维度”。

4.1 四级层次：剧本 → 场 → 镜 → 拍点

第一层是**剧本解析层**，这一层完全不用 LLM：用 Fountain/FDX 解析器把剧本文本结构化。Fountain 是 John August 等人发起的纯文本剧本标记格式，有成熟的 Python 解析库（screenplain 可直接把.fountain 转成结构化文档并导出 PDF/FDX）(Fountain, screenplain)。解析产出场标（INT./EXT.、地点、时间）、动作行、角色行、对白行，作为后续所有 LLM 调用的确定性上下文——**LLM 不应负责”读剧本格式”，只应负责”理解剧本语义”**。

第二层是**场（Scene）**：LLM 逐场产出时空设定（内/外、日/夜）、出场角色清单、所需道具清单、以及一个粗略的布景纵深描述（前/中/后景各有什么）。这一层信息密度低、稳定性高，适合一次性生成后让用户确认，确认后成为该场所有镜头共

享的”舞台”。第三层是**镜（Shot）**：LLM 把每场的动作与对白切成镜头序列，每镜给出景别、相机角度、运镜原语、时长（秒）、叙事意图。第四层是**拍点（Beat）**：每镜内部 2–5 个拍点，记录谁、做什么动作、与谁/什么道具交互、在镜头时间轴的哪个相对位置发生。

4.2 五条轨道：一个镜头契约的内部结构

每个镜头的 JSON 契约建议组织为五条并行轨道，对齐 AniMaster 验证过的 Bordwell 四子系统划分 (AniMaster)，并按白模预演的需要裁剪：

轨道	字段（LLM 输出）	求解器映射（插件生成）
Staging（布景）	场景锚点列表（门/桌/窗…）、depth_layer（fg/mid/bg）、元素间方位关系（A 在 B 左侧 2 米）	摆放白模代理几何体，求解满足方位关系的坐标
Cast Action（角色动作）	拍点表：verb（walk_to/sit/turn/look_at…）、subject、target、at（0–1 相对时机）、pace	角色代理体位移/朝向关键帧，stepped 或线性插值
Camera（相机）	shot_size、angle（eye/high/low/dutch）、move 原语 + speed + subject、framing_from→to	相机 rig 参数化：起止位置、目标 empty、焦距关键帧
Light（灯光）	time_of_day、mood、可选 key_direction	套预设灯光模板（如日光 = 单平行光 + 环境光）
Timing（节奏）	duration_sec、dialogue 行数、节奏提示	秒 → 帧换算，对齐对白占位

一个最小可行的镜头契约示例（这也是可以直接喂给结构化输出的 JSON Schema 形状）：

```
{
  "shot_id": "S01_C03",
  "intent": " 甲摊牌，乙震惊",
  "duration_sec": 6,
  "staging": {
    "anchors": [
      {"name": "door", "depth": "bg", "relation": "behind 甲"},
      {"name": "table", "depth": "mid", "relation": "between 甲 and 乙"}
    ]
  },
  "beats": [
    {"t": 0.0, "subject": " 甲", "verb": "enter_from", "target": "door", "pace": "brisk"},
    {"t": 0.4, "subject": " 甲", "verb": "walk_to", "target": "table", "face_to": " 乙"},
    {"t": 0.6, "subject": " 甲", "verb": "place_prop", "target": " 文件", "on": "table"},
    {"t": 0.7, "subject": " 乙", "verb": "look_at", "target": " 文件", "react": "shock"}
  ],
  "camera": {
    "shot_size": "MS", "angle": "eye_level",
    "move": {"primitive": "dolly_in", "speed": "slow", "subject": " 甲",
             "framing_from": "MS", "framing_to": "MCU"}
  },
  "light": {"time_of_day": "night", "mood": "tense"},
  "dialogue": [{"speaker": " 甲", "at": 0.5, "text": " 看看这个。"}]
```

```
}
```

注意这份契约里**没有任何坐标、帧号和角度数值**，全部是可以被枚举校验的语义字段；`t` 使用 0–1 归一化时机而非帧号，使契约与时长的具体换算解耦。运镜原语词表可以直接借用电影语法中成熟的分类：pan（偏航旋转）、tilt（俯仰旋转）、zoom（变焦）、dolly（前后移动）、truck（左右移动）、pedestal（升降）、tracking/follow（跟随主体）、crane/boom（大升降）、handheld（手持抖动）(Fiveable 摄影笔记, SetHero)，每一种原语在你的求解器里对应一个固定的 rig 参数化模板。

4.3 与现有系统的对照

你的设计并不是孤例，把推荐模型放到现有系统的坐标系里看会更清楚：

系统	中间表示	LLM 输出粒度	数值落地方式
SceneCraft	关系场景图（资产—空间关系二分图）	子场景描述 + 空间关系（proximity/alignment/parallelism…）	约束函数 + 求解器搜索布局参数 (arXiv)
LAMP	运动原语 DSL（原语 + 键值修饰符）	4 个运动标签/对象，相机 6DoF 原语序列	符号程序确定性映射为 3D 轨迹 (arXiv)
MovieAgent	层级 CoT（导演/编剧/分镜/选景四角色）	场景结构、相机设置、运镜的语义规划	下游视频生成模型 (GitHub)
AniMaster	Bordwell 四子系统 9 参数	景别/角度/运动/构图/灯光/色调/转场/时长/声音	交互式作者工具 (arXiv)
本报告推荐模型	**场/镜/拍点 × 五轨道 JSON 契约**	**枚举词表级语义决策**	**插件侧求解器生成关键帧**

可以看到收敛趋势非常明显：**没有任何一个成功系统让 LLM 直接产出最终数值**。中间表示的形式各不相同（图、DSL、层级规划、参数集），但”语义决策与数值落地分离”是共同架构。你的插件因为落地目标是真实的 Blender 场景而非视频生成模型，反而有一个独特优势：落地过程可以完全确定性、可回放、可调试，这是视频生成路线做不到的。

5. Blender 侧的实现映射

解构契约的另一半价值，取决于插件侧能否把语义字段稳定翻译成白模场景。这一节给出几个关键映射的工程要点。

5.1 相机：用 rig 而非裸关键帧

直接给相机对象打 location/rotation 关键帧是 LLM 生成代码时最常见也最差的做法——同时插值位置和旋转几乎必然产生漂移、穿墙和诡异的取景。Blender 社区的成熟惯例是**用空物体搭 rig**：Base empty 负责地面移动、Height empty 负责机位高度、相机本体保持水平锁定；跟随镜头用 Follow Path 曲线 + Offset 动画 + 瞄准目标，“路径决定相机在哪、Offset 决定什么时候到、瞄准决定看什么——改曲线就重塑整个运镜，一个关键帧都不用碰” (StraySpark)。瞄准统一用 Track To 约束指向一个可动画的 target empty（Blender 官方文档即”Look At”，−Z 朝目标、Y 轴向上）(Blender Manual)；已有现成插件把” 相机 + Track To 目标 + 景深焦点 empty + 可选环绕路径” 做成一键预设 (create-tracked-cameras)。

映射到你的 DSL：**每种运镜原语 = 一个 rig 模板 + 一组参数**。`dolly_in(slow, subject= 甲, MS→CU)` 翻译为：target empty 吸附到甲的胸口高度、base empty 沿视线方向从 MS 距离移动到 MCU 距离、时长由 duration_sec 换算、缓动用 ease_in_out。LLM 永远接触不到四元数、欧拉角和 F-Curve——Blender 动画数据的内部层级（Object → animation_data → Action → F-Curve → keyframe_points, 每条 F-Curve 由 data_path + array_index 寻址）只活在求解器代码里 (CGWire)。

5.2 角色与道具：代理体 + 拍点关键帧

白模阶段角色用胶囊体/方块代理即可（动作戏预演用方块正是为了速度与中性 (Kiel Figgins)）。拍点表的落地规则建议：每个拍点生成一组**位移 + 朝向**关键帧（朝向由 face_to 字段经 Track To 或阻尼追踪临时约束求解后烘焙），插值默认 stepped——这既是 blocking 的行业做法（保持" 每一键都像一张有意识的画"）(Cursa 动画教程)，也便于用户一眼看清拍点节奏。道具只在出现 place_prop/pick_up/hand_to 等交互动词时生成关键帧，并用简单的父子绑定（Child Of 约束或纯父子）跟随角色手部代理，避免道具与角色时间线脱钩。

时长换算建议用" 对白优先" 策略：有对白的镜头按时长 对白字数 ÷ 语速估算并与 LLM 给出的 duration_sec 取较大者，无对白镜头直接用 LLM 时长——动态故事板的核心经验正是" 每个镜头按真实时长停留，节奏问题在这一步暴露，而这正是 animatic 的全部价值" (Storyflow Animatic Guide)。

5.3 灯光与材质：刻意做减法

白模预演的灯光应该克制到近乎简陋：一个" 日/夜/黄昏 × 明快/阴郁" 的 3–6 套灯光预设矩阵，外加可选主光方向（顺光/侧光/逆光），足够传达时间氛围即可。给每盏灯开放强度、色温、尺寸等参数给 LLM，只会制造校验负担而不提升预演价值——预演的四个要素（板、节奏、临时音轨、可选运动）里根本没有精细灯光这一项 (Storyflow Animatic Guide)。材质同理：统一灰/白模材质，仅用明度区分角色、道具、布景三类实体。如果未来要支持灯光预演（lighting previz），再把它作为独立模块升级，而不是现在就把维度铺平。

6. 工程可靠性：契约、校验与人在环

粒度设计解决了"LLM 输出什么"，还需要一套机制解决" 输出错了怎么办"。结构化输出是地基：用 JSON Schema + strict 模式约束输出，官方评估中复杂 schema 遵循率可以从旧模型的不足 40% 提升到接近 100% (OpenAI Structured Outputs)，但要对" 接近" 保持清醒——枚举越界、属性错位仍会发生 (OpenAI 社区实测)。因此契约校验必须是双层的：**schema 层**（结构、枚举、必填）由解析器自动挡掉，**语义层**（引用的角色是否存在、target 锚点是否在本场 staging 清单里、t 是否单调递增）由你自己的校验器检查，校验失败的镜头带着错误信息回喂 LLM 局部重生成。

视觉校验回路是第二道保险。SceneCraft 用 GPT-V 看渲染图、发现错位就定位到出问题的语义关系并修订脚本，这套 critic-and-revise 回路把场景质量拉到显著超过基线的水平 (SceneCraft, ICML 2024)；BlenderMCP 类工具也把" 多视角/线框/X 光/动画抽帧截图回读，让 AI 自查" 列为核心能力 (BlenderMCP)。你的插件天然适合更轻的版本：每个镜头求解完成后自动从相机视角渲一张工作图 + 一张顶视走位图，用户（或可选的 VLM）对照契约检查，发现" 甲应该在乙右边" 这类问题时，**修改的是契约 JSON 而不是 Blender 场景**，然后局部重跑求解器——这保持了" 契约是唯一事实来源（single source of truth）"的架构纯洁性，也让每次人工干预都沉淀为可版本化的数据。

最后是契约的生命周期管理。schema 一旦定型就是 API：加必填字段、改枚举值、调整嵌套都会破坏存量数据，需要版本号、迁移路径和回归测试（用代表性剧本批量跑 shadow 验证）(TrueFoundry)。同时建议每个镜头契约保存生成时的模型、prompt 版本与校验日志，方便你在换模型或调词表后做 A/B 对比。LLM 代码生成社区的基准研究还提醒一点：反馈回路确实能显著提升质量（LLMloop 把 pass@10 从 76.22% 提到 90.24%），但 LLM 错误高度聚类、系统性偏置明显，别把" 自检通过" 当成" 正确" (LLMloop, NeurIPS 2025)——人审通道永远要留着。

7. 落地路线图建议

MVP（先验证甜区）：只做" 场 → 镜" 两层 + 三条轨道（Staging 静态摆放、Camera 运镜原语、Timing 时长）。角色只在 staging 里摆位，不做拍点动作。这一版就能产出" 会运镜的白模巡场"，足以验证契约设计、词表覆盖度和求解器的稳定性，也是用户价值密度最高的一刀——FrameForge 这类成熟预演工具的核心卖点本来就在" 结构化走位 + 光学准确的虚拟相机"，而不在角色表演 (GuideFlow)。

第二阶段：加入 Cast Action 拍点表与道具交互事件，实现 blocking 级角色运动（stepped 关键帧 + 朝向求解）。**第三阶段**：加入视觉校验回路（自动出图、VLM 可选复核）、契约版本管理与局部重生成。灯光模板矩阵可以放在任何一阶段顺手做，但始终保持" 语义字段 → 预设模板" 的间接结构。全程坚持两条纪律：一，LLM 输出的任何东西都要能被枚举校

验或引用校验，否则不进契约；二，Blender 场景永远可以从契约重新生成，不保留”游离的手工状态”（用户在 Blender 里的手动调整，要么回写到契约，要么明确标记为一次性覆写）。

值得单独一提的是架构选型：如果你的插件未来想接入通用 LLM 客户端生态，MCP 架构（Blender addon 起 socket 服务，外部 agent 经 MCP server 调用）在多步编排、模型中立、跨软件组合上优于传统自包含 addon；但如果你要的是”剧本导入 → 一键出预演”的封闭 UX，传统 addon 架构更可控 (StraySpark 架构对比)。**两种架构都与本报告的解构模型兼容——契约层在架构之外，这是它最大的设计红利**。

8. 结语

回到你最初的问题：“是否应该规划剧本实现步骤、每个分镜拆解到什么地步”——规划是必须的，而拆解的终点不是关键帧，是**决策**。把你脑子里的”角色行动关键帧、旋转、道具关键帧、前中后景、灯光、运镜”重组为”场/镜/拍点四级层次 × Staging/Action/Camera/Light/Timing 五条轨道”的语义契约，让 LLM 在枚举词表的护栏内做分镜师，让求解器在确定性代码的护栏内做执行，你就得到了一个每个环节都可校验、可复现、可人工接管、可随模型升级而升级的系统。白模预演的价值从来不在”自动”，而在”便宜地犯错”——粒度设计的全部意义，就是把犯错的位置从 Blender 场景挪到一份人类可读的 JSON 里。